Here is the complete documentation and analysis based on the provided Password Policy Validation Engine Python script.

1. Problem Statement, Objectives, and Assumptions

Problem Statement

Weak passwords compromise digital security and fail to meet institutional standards. Manually auditing passwords against security criteria can be tedious, so an automated command-line tool is required to validate whether a given password string satisfies complexity rules such as minimum length, uppercase letters, lowercase letters, and numeric digits.

Objectives

- Prompt the user to enter a sample password string.
- Iterate through the password characters to check for the presence of uppercase letters, lowercase letters, and numeric digits.
- Validate that the password length is at least 8 characters.
- Provide specific, actionable error messages for any unmet policy criteria.
- Output a final confirmation message if all criteria are successfully met.

Assumptions

- The tool operates in a single-session command-line environment.
- Password validation rules are fixed to a minimum length of 8, at least one uppercase letter, one lowercase letter, and one digit.

2. Algorithm, Flowchart, and Pseudocode

Algorithm

1. Prompt the user to enter a sample password.
2. Initialize boolean flags has_upper, has_lower, and has_digit to False.
3. Iterate through each character ch in the password:
    - If ch is uppercase, set has_upper = True.

If ch is lowercase, set has_lower = True.

-
-

If ch is a digit, set has_digit = True.

-

7.

Initialize valid = True.

8.
9.

Check length: if length of password is less than 8, print failure message and set valid = False.

10.
11.

Check has_upper: if false, print failure message and set valid = False.

12.
13.

Check has_lower: if false, print failure message and set valid = False.

14.
15.

Check has_digit: if false, print failure message and set valid = False.

16.
17.

If valid is true, print success statement; otherwise, print failure statement.

18.

Flowchart Representation

Pseudocode

3. Python Source Code and Test Cases

Python Source Code

```
                input "Enter a sample password: "
                 False
                 False
                False
for     in
  if
                    True
  if
                    True
  if
                   True

        Trueif len                  8
        "Failed: must contain at least 8 characters."
           Falseif not
        "Failed: add an uppercase letter."
           Falseif not
        "Failed: add a lowercase letter."
           Falseif not
```

```
        "Failed: add a digit."
          False
if
        "Password meets the classroom rules." else
        "Password does not meet the rules."
```

(Source code adapted from).

Test Cases & Sample Output

Test Case ID	Scenario Description	User Inputs	Expected Output / Behavior
TC-01	Fully Compliant Password	Python123	Password meets the classroom rules.
TC-02	Short Password (Missing Length)	Py1	Fails length check; outputs failure messages for length, uppercase/lowercase/digit depending on string.
TC-03	Missing Numeric Digit	Pythonistas	Fails digit check; outputs "Failed: add a digit".

Simulated Terminal Output (for TC-01)

4. Conclusion

The password policy validation engine automates security compliance checks by analyzing string characteristics iteratively. It provides clear, specific feedback to users regarding missing criteria, reinforcing robust authentication practices efficiently.